PUT & DELETE in FastAPI

Now that you understand:
GET → Retrieve data
POST → Create data
The next step is:
PUT → Update data
DELETE → Remove data
Together, these complete the **CRUD operations**:

CRUD Operation	HTTP Method
Create	POST
Read	GET
Update	PUT
Delete	DELETE

---

1. **Introduction**

Imagine a Student Management System.
Initially:

```
[
 {
  "id": 1,
  "name": "Ali",
  "age": 20
 }
]
```

Now you want to:

1. Change Ali's age → UPDATE
2. Remove Ali completely → DELETE

That's where PUT and DELETE come in.

What is PUT?
Definition
PUT is used to **update an existing resource**.
Think:
"I already have this data. I want to replace it with new data."

Real-Life Examples
Student System
Current:

```
{
 "id": 1,
 "name": "Ali",
```

```
  "age": 20
}
```

Update to:

```
{
  "id": 1,
  "name": "Ali",
  "age": 21
}
```

E-commerce

Update product details:

```
{
  "name": "Laptop",
  "price": 1000
}
```

to

```
{
  "name": "Laptop",
  "price": 900
}
```

PUT Request Structure

PUT usually uses:

1. Path Parameter → identify resource
2. Request Body → new data

Example URL

```
/students/1
```

Student ID = 1

Request Body

```
{
  "name": "Ali",
  "age": 21
}
```

FastAPI PUT Example

Step 1: Create Model

```
from pydantic import BaseModel

class Student(BaseModel):
    name: str
```

```
    age: int
```

Step 2: Sample Database

```
students = {
    1: {"name": "Ali", "age": 20}
}
```

Step 3: PUT Endpoint

```
@app.put("/students/{student_id}")
def update_student(student_id: int, student: Student):
    students[student_id] = student.model_dump()

    return {
        "message": "Student updated",
        "student": students[student_id]
    }
```

What Happens?

Request:

```
PUT /students/1
```

Body:

```
{
  "name": "Ali",
  "age": 21
}
```

Response:

```
{
  "message": "Student updated",
  "student": {
    "name": "Ali",
    "age": 21
  }
}
```

Understanding PUT Visually

Before:

```
{
  "name": "Ali",
  "age": 20
}
```

After PUT:

```
{
  "name": "Ali",
  "age": 21
}
```

Old record is replaced by new record.

PUT vs POST

Many beginners confuse these.

POST	PUT
Create new data	Update existing data
New resource	Existing resource
Add record	Modify record

Example

POST:

POST /students

Creates:

```
{
  "id": 10,
  "name": "Ahmed"
}
```

PUT:

```
PUT /students/10
```

Updates:

```
{
  "id": 10,
  "name": "Ahmed Khan"
}
```

Important: PUT Usually Replaces Entire Resource

Suppose student is:

```
{
  "name": "Ali",
  "age": 20,
  "department": "CS"
}
```

PUT request:

```
{
  "name": "Ali Khan"
}
```

In strict REST design, PUT means:

Replace the whole resource.
This is why PATCH exists for partial updates.

What is DELETE?
Definition
DELETE removes an existing resource.
Think:
"This data is no longer needed."

Real-Life Examples
Student System
Delete student:

```
/students/5
```

E-commerce
Delete product:

```
/products/100
```

Todo App
Delete task:

```
/tasks/3
```

FastAPI DELETE Example

Sample Data

```
students = {
  1: {"name": "Ali"},
  2: {"name": "Ahmed"}
}
```

DELETE Endpoint

```
@app.delete("/students/{student_id}")
def delete_student(student_id: int):

  del students[student_id]

  return {
    "message": "Student deleted"
  }
```

Request

```
DELETE /students/1
```

Response

```
{
  "message": "Student deleted"
}
```

What Happens Internally?

Before:

```
{
  "1": {"name": "Ali"},
  "2": {"name": "Ahmed"}
}
```

DELETE:

```
DELETE /students/1
```

After:

```
{
  "2": {"name": "Ahmed"}
}
```

Student 1 no longer exists.

Handling Missing Data

Suppose user tries:

```
DELETE /students/100
```

But student 100 doesn't exist.

FastAPI should return:

```
from fastapi import HTTPException

@app.delete("/students/{student_id}")
def delete_student(student_id: int):

    if student_id not in students:
        raise HTTPException(
            status_code=404,
            detail="Student not found"
        )

    del students[student_id]

    return {"message": "Deleted"}
```

Response:

```
{
  "detail": "Student not found"
}
```

Status Code:

```
404 Not Found
```

HTTP Status Codes for PUT & DELETE

PUT

Successful update:
200 OK
or
204 No Content

DELETE

Successful deletion:
200 OK
or
204 No Content

Resource not found:
404 Not Found

---

3. Complete CRUD Example

```
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()
students = {}

class Student(BaseModel):
  name: str
  age: int

@app.post("/students/{id}")
def create_student(id: int, student: Student):
  students[id] = student.model_dump()
  return students[id]

@app.get("/students/{id}")
def get_student(id: int):
  return students[id]

@app.put("/students/{id}")
def update_student(id: int, student: Student):
```

```
    students[id] = student.model_dump()
    return students[id]

@app.delete("/students/{id}")
def delete_student(id: int):
    del students[id]
    return {"message": "Deleted"}
```

Machine Learning Example
Suppose you deploy a sentiment analysis model.

Create Model
POST /models

View Model
GET /models/1

Update Model Settings
PUT /models/1
Body:

```
{
  "threshold": 0.8
}
```

Remove Model
DELETE /models/1

Action	HTTP Method
Add page	POST
Read page	GET
Rewrite page	PUT
Tear out page	DELETE

POST
Creates new data.
POST /students

GET
Retrieves data.
GET /students/1

PUT
Updates existing data.

PUT /students/1
Uses:

- Path Parameter (which record?)
- Request Body (new data)

DELETE
Removes existing data.
DELETE /students/1
Uses:

- Path Parameter (which record?)

These four methods (GET, POST, PUT, DELETE) form the core of almost every FastAPI backend, REST API, and Machine Learning service you'll build.